```
Start coding or generate with AI.
```

```
!pip install nltk

import nltk
nltk.download('brown')
nltk.download('universal_tagset')
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Unzipping corpora/brown.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Unzipping taggers/universal_tagset.zip.
True
```

```
sentences = brown.sents()
print("Total Sentences:", len(sentences))
```

```
Total Sentences: 57340
```

```
tokens = [word.lower() for sent in sentences for word in sent]
print("Total Tokens:", len(tokens))
```

```
Total Tokens: 1161192
```

```
tagged_sentences = brown.tagged_sents(tagset='universal')

# Convert words to lowercase
tagged_sentences = [[(word.lower(), tag) for word, tag in sent]
                    for sent in tagged_sentences]

print("Sample Tagged Sentence:")
print(tagged_sentences[0])
```

```
Sample Tagged Sentence:
[('the', 'DET'), ('fulton', 'NOUN'), ('county', 'NOUN'), ('grand', 'ADJ'), ('jury', 'NOUN'), ('said', 'V
```

```
from nltk.tag import hmm
trainer = hmm.HiddenMarkovModelTrainer()
hmm_model = trainer.train_supervised(tagged_sentences)

print("HMM Model Trained Successfully!")
```

```
HMM Model Trained Successfully!
```

```
from collections import defaultdict, Counter

transition_counts = defaultdict(Counter)

for sent in tagged_sentences:
    for i in range(len(sent)-1):
        prev_tag = sent[i][1]
        next_tag = sent[i+1][1]
        transition_counts[prev_tag][next_tag] += 1

print("Sample POS Transitions:")
```

```
print(transition_counts['NOUN'].most_common(5))
```

```
Sample POS Transitions:
[('.', 78181), ('ADP', 67325), ('VERB', 43763), ('NOUN', 41144), ('CONJ', 16447)]
```

```
emission_counts = defaultdict(Counter)

for sent in tagged_sentences:
    for word, tag in sent:
        emission_counts[tag][word] += 1

print("Sample Emissions for VERB:")
print(emission_counts['VERB'].most_common(5))
```

```
Sample Emissions for VERB:
[('is', 10108), ('was', 9815), ('be', 6374), ('had', 5132), ('are', 4394)]
```

```
def predict_next_word(previous_word):

    # Get POS tag of previous word
    prev_tag = hmm_model.tag([previous_word.lower()])[0][1]

    # Get most likely next POS tag
    if prev_tag not in transition_counts:
        return []

    next_tag = transition_counts[prev_tag].most_common(1)[0][0]

    # Get most common word emitted by that POS
    suggestions = emission_counts[next_tag].most_common(5)

    return [word for word, _ in suggestions]
```

```
print("Prediction after 'government':")
print(predict_next_word("government"))

print("\nPrediction after 'the':")
print(predict_next_word("the"))

print("\nPrediction after 'he':")
print(predict_next_word("he"))
```

```
Prediction after 'government':
/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:335: RuntimeWarning: overflow encountered in cas
  O[i, k] = self._output_logprob(si, self._symbols[k])
[',', '.', '``', "''", ';']

Prediction after 'the':
['time', 'man', 'af', 'years', 'way']

Prediction after 'he':
['is', 'was', 'be', 'had', 'are']
```